1. Goal of Backpropagation:

by using Gradient Descent Method on multilayer neural network in order to find out appropriate weights that can minimize value of error function ($\equiv \hat{y}_z - \vec{y}_z \longrightarrow 0$)

2. Flow of Backpropagation:

A. forward pass

B. backward pass

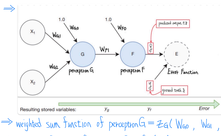
C. weight adjustment

3. Applying Backpropagation on Two-Layer Neural Network:

A.

B. forward pass:

⇒



⇒ weighted sum function of perceptron G = $z_{G1}(W_{G0}, W_{G1}, W_{G2})$

⇒ activation function of perceptron G = $f_{G1}(z_{G1})$

⇒ weighted sum function of perceptron F = $z_F(W_{F0}, W_{F1}, y_{G1})$

⇒ activation function of perceptron F = $f_F(z_F)$

⇒ predicted output $\hat{y}_z = \hat{y}_z = f_F(W_{F0} \cdot 1 + W_{F1} f_{G1}(W_{G0} \cdot 1 + W_{G1} X_1 + W_{G2} X_2))$

⇒ error function = e = Mean Square Error $(MSE) = \frac{1}{m}\sum_{i=0}^{m}(\hat{y}_i - \vec{y}_i)^2$, [$\hat{y}_i$ = desire output (ground truth) from i-th pair training example which corresponding to single output perceptron; $\vec{y}_i$ = predicted output $\hat{y}_z$ from single output perceptron which corresponding to i-th pair training example; m = number of pair trainings examples]

⇒ error function = $e(\vec{y}) = e(\hat{y}_z) = \frac{1}{2}(\hat{y}_z - \vec{y}_z)^2$ [$\because \hat{y} = \hat{y}_z$]

⇒ error function = $e(\vec{y}) = e(W_{F0}, W_{F1}, W_{G0}, W_{G1}) = e \cdot f_F \cdot z_F \cdot \hat{y}_{G1} \cdot z_{G1}(W_{G0}, W_{G1}, W_{G2})$

⇒ error function = $e(\vec{y}) = e(W_{F0}, W_{F1}, W_{G0}, W_{G1}) = e \cdot f_F \cdot z_F(W_{F0}, W_{F1})$

⇒ $\frac{\partial e}{\partial W_{G0}} = \frac{\partial e}{\partial \hat{y}_z}\frac{\partial \hat{y}_z}{\partial f_F}\frac{\partial f_F}{\partial z_F}\frac{\partial z_F}{\partial y_{G1}}\frac{\partial y_{G1}}{\partial z_{G1}}\frac{\partial z_{G1}}{\partial W_{G0}}$

⇒ $\frac{\partial e}{\partial W_{G1}} = \frac{\partial e}{\partial \hat{y}_z}\frac{\partial \hat{y}_z}{\partial f_F}\frac{\partial f_F}{\partial z_F}\frac{\partial z_F}{\partial y_{G1}}\frac{\partial y_{G1}}{\partial z_{G1}}\frac{\partial z_{G1}}{\partial W_{G1}}$

⇒ $\frac{\partial e}{\partial W_{G2}} = \frac{\partial e}{\partial \hat{y}_z}\frac{\partial \hat{y}_z}{\partial f_F}\frac{\partial f_F}{\partial z_F}\frac{\partial z_F}{\partial y_{G1}}\frac{\partial y_{G1}}{\partial z_{G1}}\frac{\partial z_{G1}}{\partial W_{G2}}$

⇒ $\frac{\partial e}{\partial W_{F0}} = \frac{\partial e}{\partial \hat{y}_z}\frac{\partial \hat{y}_z}{\partial f_F}\frac{\partial f_F}{\partial z_F}\frac{\partial z_F}{\partial W_{F0}}$

⇒ $\frac{\partial e}{\partial W_{F1}} = \frac{\partial e}{\partial \hat{y}_z}\frac{\partial \hat{y}_z}{\partial f_F}\frac{\partial f_F}{\partial z_F}\frac{\partial z_F}{\partial W_{F1}}$

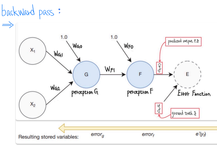
⇒ Gradient Descent Method: $\vec{W} = \vec{W} + \eta \cdot (-\nabla e)$

⇒ Gradient Descent Method: $\begin{bmatrix} w_{G0} \\ w_{G1} \\ w_{G2} \\ w_{F0} \\ w_{F1} \end{bmatrix} = \begin{bmatrix} w_{G0} \\ w_{G1} \\ w_{G2} \\ w_{F0} \\ w_{F1} \end{bmatrix} + \eta \cdot \left(- \begin{bmatrix} \frac{\partial e}{\partial W_{G0}} \\ \frac{\partial e}{\partial W_{G1}} \\ \frac{\partial e}{\partial W_{G2}} \\ \frac{\partial e}{\partial W_{F0}} \\ \frac{\partial e}{\partial W_{F1}} \end{bmatrix} \right)$

not this today, just theoretical derivation

C. backward pass:

⇒



⇒ compare e partial derivate to $\hat{y}_z$ to get Overall Error:

⇒ $e(\vec{y}) = e(\hat{y}_z) = \frac{1}{2}(\hat{y}_z - \vec{y}_z)^2 = (\hat{y}_z - \vec{y}_z)^1$ [$\because \hat{y} = \hat{y}_z$]

⇒ Overall Error = $e'(\hat{y}_z) = \frac{\partial e}{\partial \hat{y}_z} = -2(\vec{y} - \hat{y}_z)$ [$\because \hat{y} = \hat{y}_z$]

⇒ compare e partial derivate to weighted sum function of each perceptron in every layers to get Error Term:

⇒ $Error_F = \frac{\partial e}{\partial z_F} = \left(\frac{\partial e}{\partial \hat{y}_z}\right)\frac{\partial \hat{y}_z}{\partial z_F} = (\text{Overall Error}) \cdot f_F'(z_F)$

⇒ $Error_G = \frac{\partial e}{\partial z_{G1}} = \left(\frac{\partial e}{\partial \hat{y}_z}\frac{\partial \hat{y}_z}{\partial z_F}\right)\frac{\partial z_F}{\partial y_{G1}}\frac{\partial y_{G1}}{\partial z_{G1}} = (Error_F) \cdot W_{F1} \cdot f_{G1}'(z_{G1})$

D. weight adjustment:

⇒ Gradient Descent Method: $\vec{W} = \vec{W} + \eta \cdot (-\nabla e)$

⇒ Gradient Descent Method: $\begin{bmatrix} w_{G0} \\ w_{G1} \\ w_{G2} \\ w_{F0} \\ w_{F1} \end{bmatrix} = \begin{bmatrix} w_{G0} \\ w_{G1} \\ w_{G2} \\ w_{F0} \\ w_{F1} \end{bmatrix} + \eta \cdot \left(- \begin{bmatrix} \frac{\partial e}{\partial W_{G0}} \\ \frac{\partial e}{\partial W_{G1}} \\ \frac{\partial e}{\partial W_{G2}} \\ \frac{\partial e}{\partial W_{F0}} \\ \frac{\partial e}{\partial W_{F1}} \end{bmatrix} \right)$

⇒ Gradient Descent Method: $\begin{bmatrix} w_{G0} \\ w_{G1} \\ w_{G2} \\ w_{F0} \\ w_{F1} \end{bmatrix} = \begin{bmatrix} w_{G0} \\ w_{G1} \\ w_{G2} \\ w_{F0} \\ w_{F1} \end{bmatrix} + \eta \cdot \left(- \begin{bmatrix} \frac{\partial e}{\partial \hat{y}_z}\frac{\partial \hat{y}_z}{\partial f_F}\frac{\partial f_F}{\partial z_F}\frac{\partial z_F}{\partial y_{G1}}\frac{\partial y_{G1}}{\partial z_{G1}}\frac{\partial z_{G1}}{\partial W_{G0}} \\ \frac{\partial e}{\partial \hat{y}_z}\frac{\partial \hat{y}_z}{\partial f_F}\frac{\partial f_F}{\partial z_F}\frac{\partial z_F}{\partial y_{G1}}\frac{\partial y_{G1}}{\partial z_{G1}}\frac{\partial z_{G1}}{\partial W_{G1}} \\ \frac{\partial e}{\partial \hat{y}_z}\frac{\partial \hat{y}_z}{\partial f_F}\frac{\partial f_F}{\partial z_F}\frac{\partial z_F}{\partial y_{G1}}\frac{\partial y_{G1}}{\partial z_{G1}}\frac{\partial z_{G1}}{\partial W_{G2}} \\ \frac{\partial e}{\partial \hat{y}_z}\frac{\partial \hat{y}_z}{\partial f_F}\frac{\partial f_F}{\partial z_F}\frac{\partial z_F}{\partial W_{F0}} \\ \frac{\partial e}{\partial \hat{y}_z}\frac{\partial \hat{y}_z}{\partial f_F}\frac{\partial f_F}{\partial z_F}\frac{\partial z_F}{\partial W_{F1}} \end{bmatrix} \right)$

⇒ Gradient Descent Method: $\begin{bmatrix} w_{G0} \\ w_{G1} \\ w_{G2} \\ w_{F0} \\ w_{F1} \end{bmatrix} = \begin{bmatrix} w_{G0} \\ w_{G1} \\ w_{G2} \\ w_{F0} \\ w_{F1} \end{bmatrix} + \eta \cdot \left(- \begin{bmatrix} Error_G \cdot \frac{\partial z_{G1}}{\partial W_{G0}} \\ Error_G \cdot \frac{\partial z_{G1}}{\partial W_{G1}} \\ Error_G \cdot \frac{\partial z_{G1}}{\partial W_{G2}} \\ Error_F \cdot \frac{\partial z_F}{\partial W_{F0}} \\ Error_F \cdot \frac{\partial z_F}{\partial W_{F1}} \end{bmatrix} \right)$

⇒ Gradient Descent Method: $\begin{bmatrix} w_{G0} \\ w_{G1} \\ w_{G2} \\ w_{F0} \\ w_{F1} \end{bmatrix} = \begin{bmatrix} w_{G0} \\ w_{G1} \\ w_{G2} \\ w_{F0} \\ w_{F1} \end{bmatrix} + \eta \cdot \left(- \begin{bmatrix} Error_G \cdot 1.0 \\ Error_G \cdot X_1 \\ Error_G \cdot X_2 \\ Error_F \cdot 1.0 \\ Error_F \cdot f_{G1} \end{bmatrix} \right)$

⇒ Gradient Descent Method: $\vec{W} = \vec{W} + \eta \cdot (-$ "Error Term of the weight's perceptron" $\cdot$ "Input of the weight"$)$

3. Applying Backpropagation on Two-Layer Neural Network with Real Data:

A. real data:

⇒ $w_{G0} = 0.3$, $w_{G1} = 0.6$, $w_{G2} = -0.1$, $w_{F0} = -0.2$, $w_{F1} = 0.5$

⇒ $X_1 = -0.9$, $X_2 = 0.1$,

⇒ ground truth $(\vec{y}) = 1.0$

⇒ $\eta = 0.05$

⇒ $f_{G1} = \tanh()$, $f_F = S()$

B. forward pass:

⇒ $f_{G1} = \tanh(z_{G1}) = \tanh(W_{G0} \cdot 1 + W_{G1} X_1 + W_{G2} X_2) = \tanh(0.3 \cdot 1 + 0.6 \cdot (-0.9) + (-0.1) \cdot 0.1) = -0.25$

⇒ $f_F = S(z_F) = S(W_{F0} \cdot 1 + W_{F1} f_{G1}) = S(-0.2 \cdot 1 + 0.5 \cdot (-0.25)) = 0.42$

⇒ $e = MSE = (\hat{y} - \vec{y}_z)^2 = (1.0 - 0.42)^2 = 0.34$

C. backward pass:

⇒ Overall Error $= e'(\hat{y}_z) = (MSE)' = -2(\vec{y} - \hat{y}_z) = -2(1.0 - 0.42) = -1.16$

⇒ $Error_F = (\text{Overall Error}) \cdot f_F'(z_F) = -1.16 \cdot S'(z_F) = -1.16 \cdot S(z_F) \cdot (1 - S(z_F)) = -1.16 \cdot 0.42 \cdot (1 - 0.42) = -0.28$

⇒ $Error_G = (Error_F) \cdot W_{F1} \cdot f_{G1}'(z_{G1}) = -0.28 \cdot 0.5 \cdot (1 - \tanh^2(z_{G1})) = -0.28 \cdot 0.5 \cdot (1 - (-0.25)^2) = -0.132$

D. weight adjustment:

⇒ Gradient Descent Method: $\begin{bmatrix} w_{G0} \\ w_{G1} \\ w_{G2} \\ w_{F0} \\ w_{F1} \end{bmatrix} = \begin{bmatrix} w_{G0} \\ w_{G1} \\ w_{G2} \\ w_{F0} \\ w_{F1} \end{bmatrix} + \eta \cdot \left(- \begin{bmatrix} Error_G \cdot 1.0 \\ Error_G \cdot X_1 \\ Error_G \cdot X_2 \\ 1.0 \\ Error_F \cdot f_{G1} \end{bmatrix} \right) Error_F$

⇒ $w_{G0} = 0.3 + 0.05 \cdot -(-0.132) \cdot (1.0) = 0.3066$

⇒ $w_{G1} = 0.6 + 0.05 \cdot -(-0.132) \cdot (-0.9) = 0.5940$

⇒ $w_{G2} = -0.1 + 0.05 \cdot -(-0.132) \cdot (0.1) = -0.09735$

⇒ $w_{F0} = -0.2 + 0.05 \cdot -(-0.28) \cdot (1.0) = -0.1859$

⇒ $w_{F1} = 0.5 + 0.05 \cdot -(-0.28) \cdot (-0.25) = 0.4965$

E.

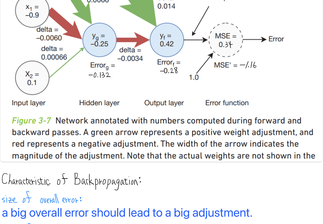


Figure 3-7 Network annotated with numbers computed during forward and backward passes. A green arrow represents a positive weight adjustment, and red represents a negative adjustment. The width of the arrow indicates the magnitude of the adjustment. Note that the actual weights are not shown in the

4. Characteristic of Backpropagation:

A. size of overall error:

a big overall error should lead to a big adjustment.

B. size of input:

if the input to the weight is small, then adjusting the weight is not helpful because it doesn't have much of an effect.

C. times of computations in Backward Pass:

the number of computing the entire gradient in one Backward Pass
is about the same as the number of computations in one Forward Pass.
This can be compared with the N+1 times the Forward Pas that would have been needed,
if we computed the gradient numerically using the brute-force method instead of backpropagation

D. sign of input and overall error:

"sign of the input values that feed each weight" and "sign of overall error value " will decide the direction that the weight is adjusted.
For example, overall error > 0 (predict output is smaller than ground truth, we need to increase predict output),
the bias weight is increased because x0 input is a postive value, whereas the weight w1 is decreased because the x1 input is a negative value.

E. a specific weight on the path:

if there is a "weight(w3) whose partial derivative approaches 0 (neural saturation)" or "its value is too small",
on the path of the weight(w2) you want to adjust and overall error in the end of the neural network,
then it is not helpful to change the weight(w2) because a weight(w3) suppress the effect of this weight(w2) change.